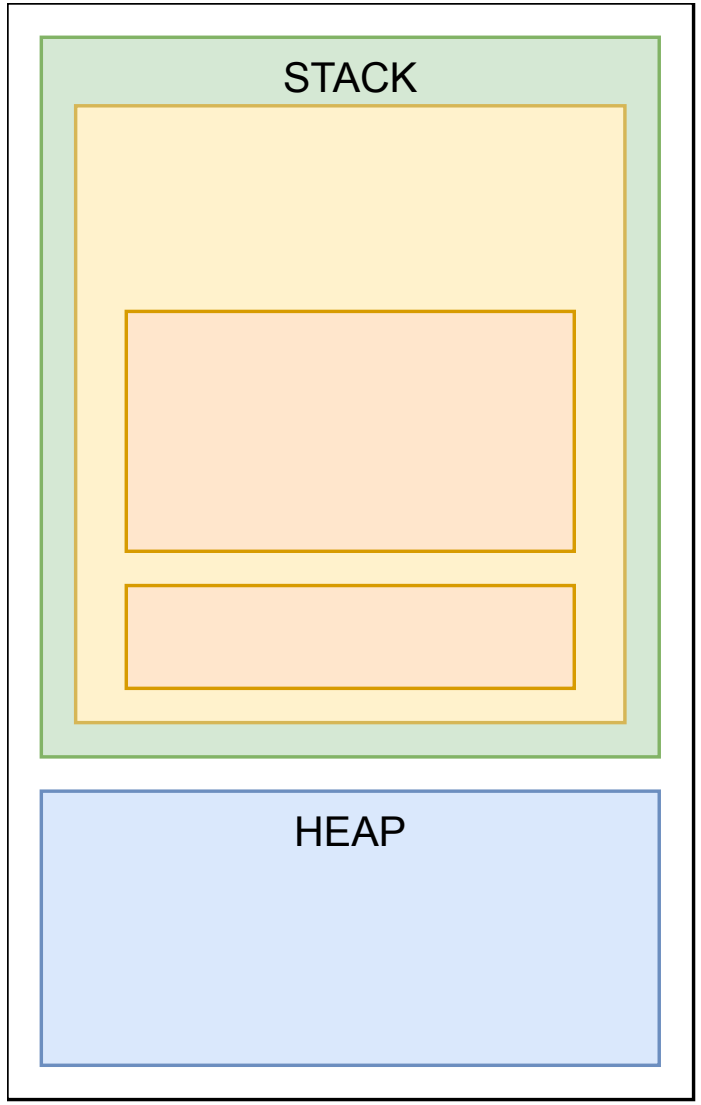
C++ - Allocation mémoire et structure RAII

BTS CIEL

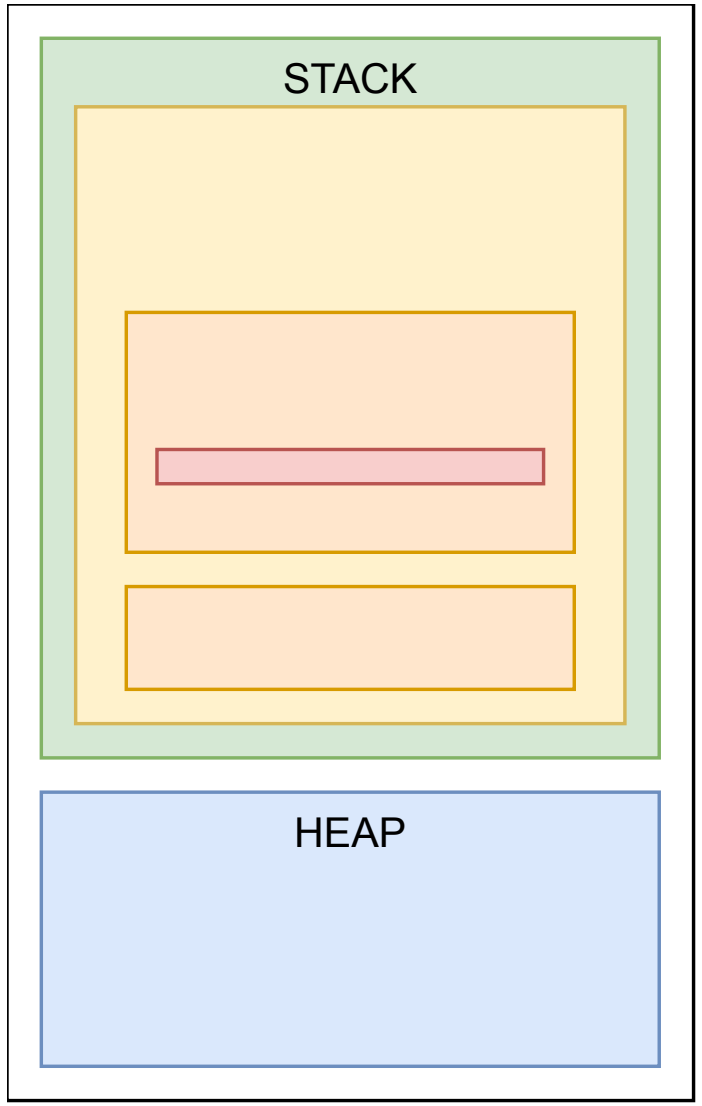
Rappel stack VS heap en C

```c
void add(int a, int b)
{
  return a + b;
}

int main()
{
  int a = 10;
  int b = 12;

  int result = add(a, b);

  printf("Result: %d", result);

  return 0;
}
```



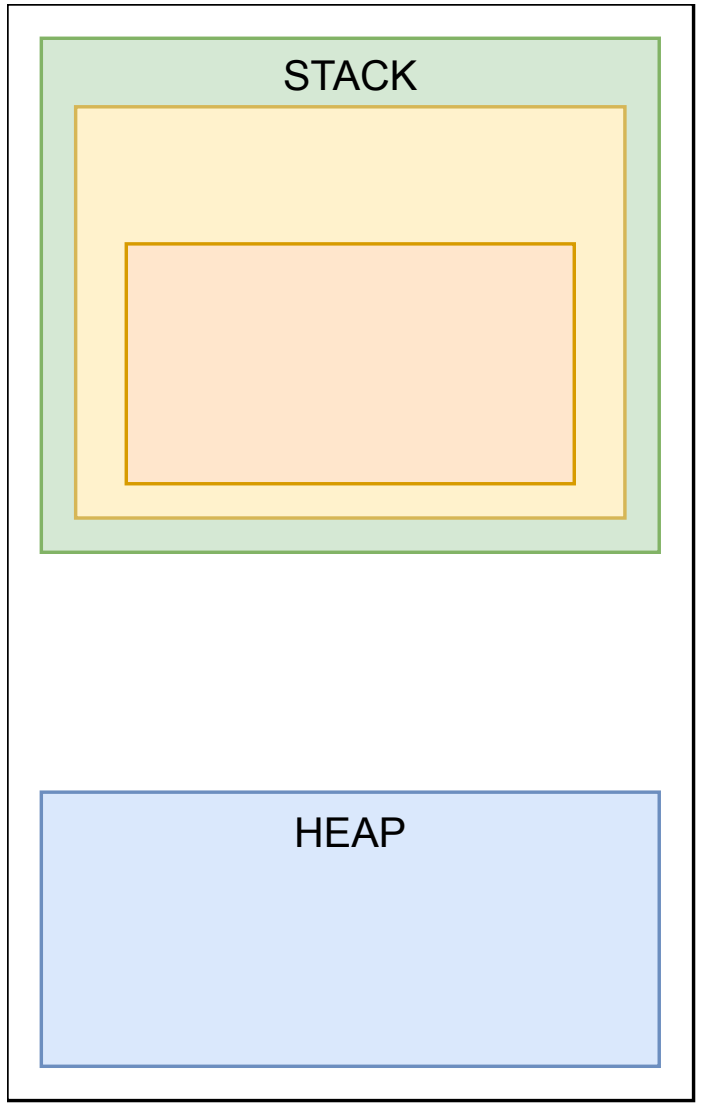
Rappel stack VS heap en C

```c
typedef struct Point {int x; int y;} Point;

Point* new_point_from(Point a, Point b)
{
  Point* p = malloc(sizeof(Point));

  if (p == NULL) return NULL;

  p->x = a.x + b.x;
  p->y = a.y + b.y;

  return p;
}

int main()
{
  Point a = {10, 10};
  Point b = {20, 20};
  Point* p = new_point_from(a, b);
  free(p);
}
```



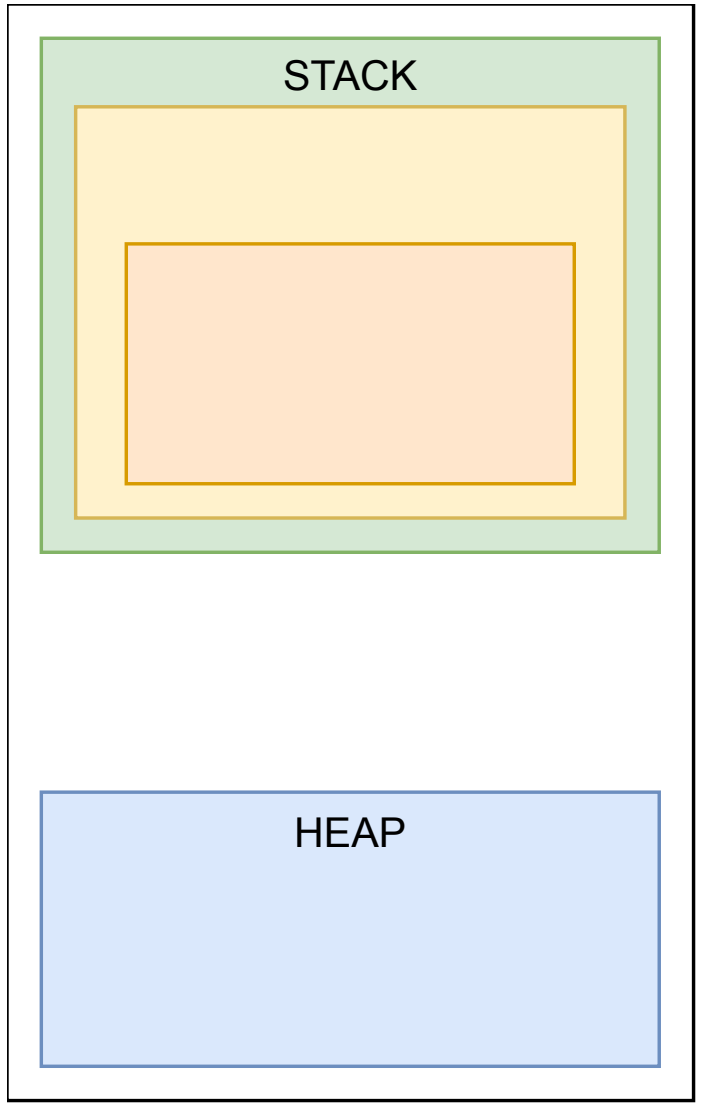
C++ new et delete

```cpp
struct Point {int x; int y;};

Point* new_point_from(Point a, Point b)
{
  Point* p = new Point();

  p->x = a.x + b.x;
  p->y = a.y + b.y;

  return p;
}

int main()
{
  Point a = {10, 10};
  Point b = {20, 20};
  Point* p = new_point_from(a, b);
  delete p;
}
```



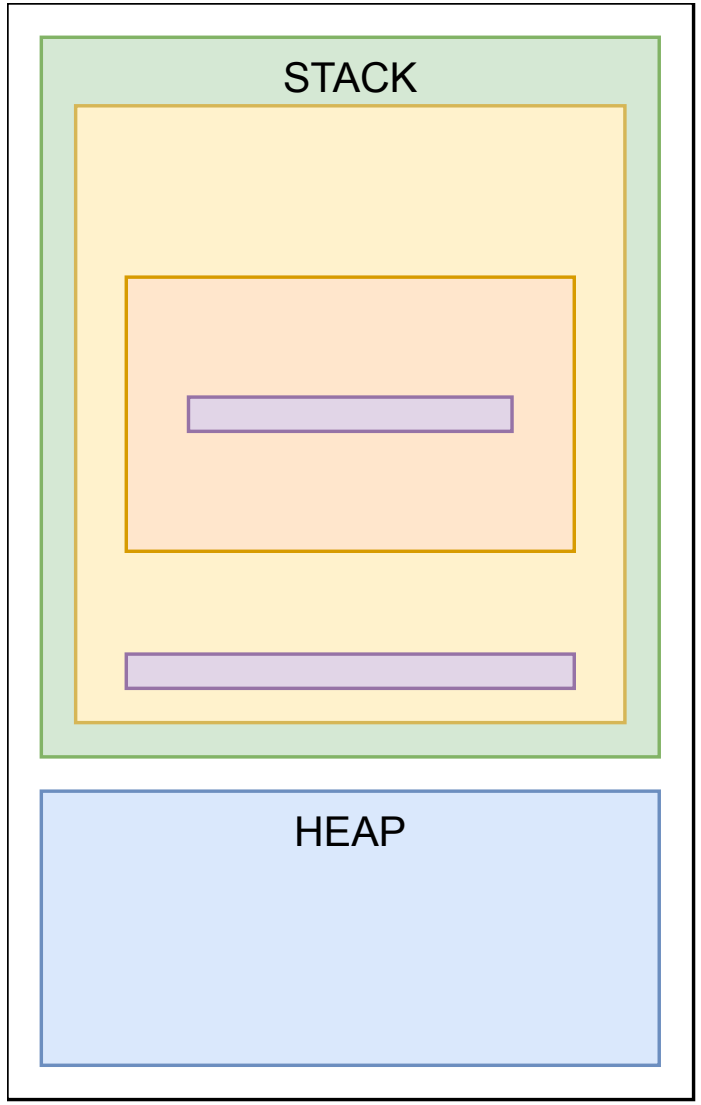
C++ référence

```cpp
struct Point {int x; int y;};

Point* new_point_from(Point& a, Point& b)
{
  Point* p = new Point();

  p->x = a.x + b.x;
  p->y = a.y + b.y;

  return p;
}

int main()
{
  Point a = {10, 10};
  Point b = {20, 20};
  Point* p = new_point_from(a, b);
  delete p;
}
```



C++ constructeur et destructeur

```cpp
struct Point {
  int x; int y;

  Point(int _x, int _y) : x(_x), y(_y) {}
  ~Point() {}
};

Point* new_point_from(Point& a, Point& b)
{
  Point* p = new Point(0, 0);

  p->x = a.x + b.x;
  p->y = a.y + b.y;

  return p;
}

int main()
{
  Point a = {10, 10};
  Point b = {20, 20};
  Point* p = new_point_from(a, b);
  delete p;
}
```



C++ RAII

```cpp
struct Point {int x; int y;};

int main()
{
  std::vector<Point> points = {};

  points.push_back(Point(10, 5));
  points.push_back(Point(20, 13));
}
```